

International Cybersecurity and
Digital Forensics Academy (ICDFA)

CIP-B101 Lab 2A

Linux Command Line for Digital Forensics Basics

Student Practical Laboratory Report

Student Name: Almustapha Yusuf

Course: CIP-B101 Basic Computer Skills for Digital Forensics

Date: 16 August 2026

Table of Contents

1. Lab Objectives and Scenario
2. Task 1: Evidence Workspace Preparation
3. Task 2: Linux Filesystem and OS Information
4. Task 3: Inodes and Disk Information
5. Task 4: Absolute and Relative Paths
6. Task 5: File Management Operations
7. Task 6: Searching Evidence with grep
8. Task 7: Directory Tree and File Discovery
9. Task 8: Network Information (Localhost)
10. Task 9: Forensic Shell Script
11. Task 10: PATH Variable and Script Accessibility
12. Task 11: Evidence Archiving
13. Report Questions and Answers
14. Conclusion

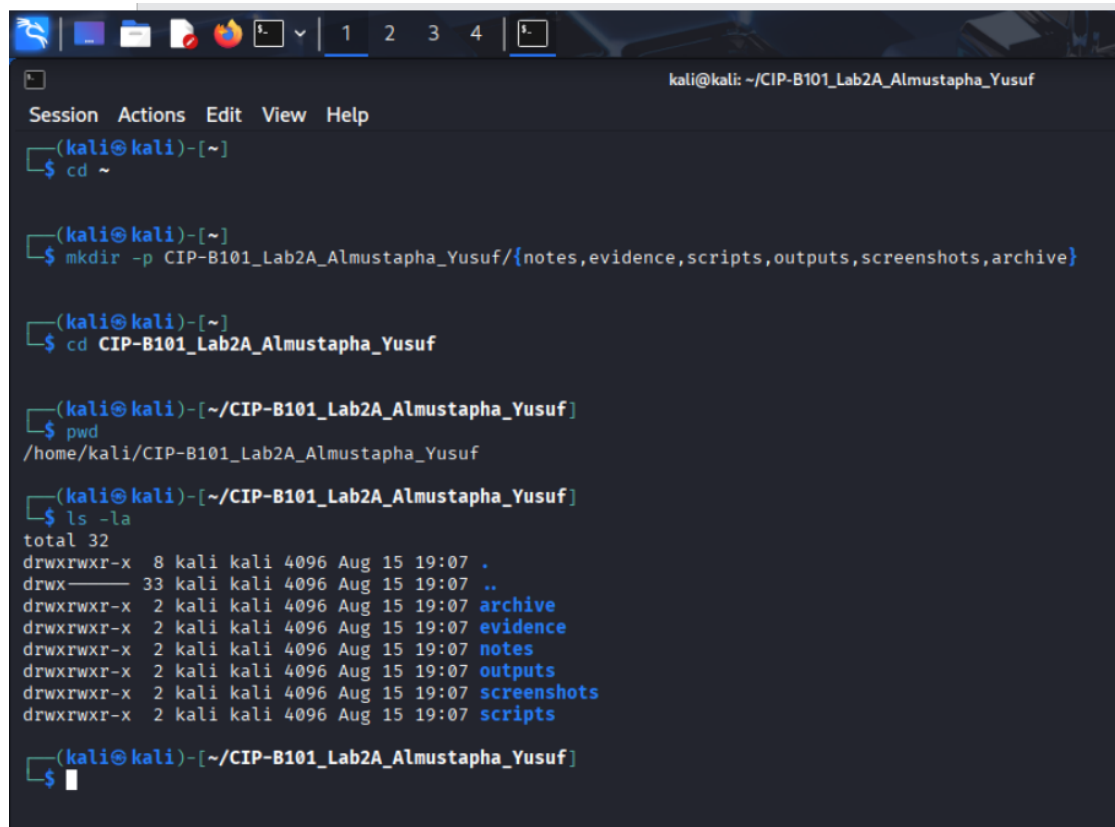
1. Lab Objectives and Scenario

This laboratory exercise focuses on fundamental Linux command-line skills that form the foundation for digital forensic work. The scenario places the student in the role of a junior digital forensic trainee tasked with preparing a Linux-based evidence workspace. The lab covers essential competencies including navigating the Linux filesystem hierarchy, understanding inode metadata, managing files and directories, searching evidence content with grep, checking network configuration, writing shell scripts, and packaging evidence for submission. These skills are directly applicable to real-world forensic operations where command-line proficiency is essential for tasks such as disk imaging, file carving, log analysis, and automated evidence processing. Linux is the preferred operating system for digital forensics because it provides direct access to file systems, supports a vast array of forensic tools, and allows investigators to work with raw disk data without the limitations imposed by graphical operating systems. Understanding the Linux command line is therefore a prerequisite for any forensic professional, and this lab provides structured hands-on practice with the most commonly used commands.

2. Task 1: Evidence Workspace Preparation

A well-organized evidence workspace is critical in digital forensics. Before any evidence collection or analysis begins, the investigator must establish a structured directory hierarchy that separates different types of artifacts. This practice mirrors professional forensic labs where evidence files, analysis notes, scripts, and output logs are kept in designated locations to maintain chain of custody, prevent cross-contamination, and ensure reproducibility of results. The workspace created in this task includes directories for notes (investigator observations), evidence (original evidence files), scripts (automation tools), outputs (command results and logs), screenshots (visual documentation), and archive (compressed submissions).

```
mkdir -p CIP-B101_Lab2A_Almustapha_Yusuf/{notes,evidence,scripts,outputs,screenshots,archive}
```



```
kali@kali: ~/CIP-B101_Lab2A_Almustapha_Yusuf
Session Actions Edit View Help
(kali@kali)-[~]
$ cd ~

(kali@kali)-[~]
$ mkdir -p CIP-B101_Lab2A_Almustapha_Yusuf/{notes,evidence,scripts,outputs,screenshots,archive}

(kali@kali)-[~]
$ cd CIP-B101_Lab2A_Almustapha_Yusuf

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ pwd
/home/kali/CIP-B101_Lab2A_Almustapha_Yusuf

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ ls -la
total 32
drwxrwxr-x  8 kali kali 4096 Aug 15 19:07 .
drwx----- 33 kali kali 4096 Aug 15 19:07 ..
drwxrwxr-x  2 kali kali 4096 Aug 15 19:07 archive
drwxrwxr-x  2 kali kali 4096 Aug 15 19:07 evidence
drwxrwxr-x  2 kali kali 4096 Aug 15 19:07 notes
drwxrwxr-x  2 kali kali 4096 Aug 15 19:07 outputs
drwxrwxr-x  2 kali kali 4096 Aug 15 19:07 screenshots
drwxrwxr-x  2 kali kali 4096 Aug 15 19:07 scripts

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$
```

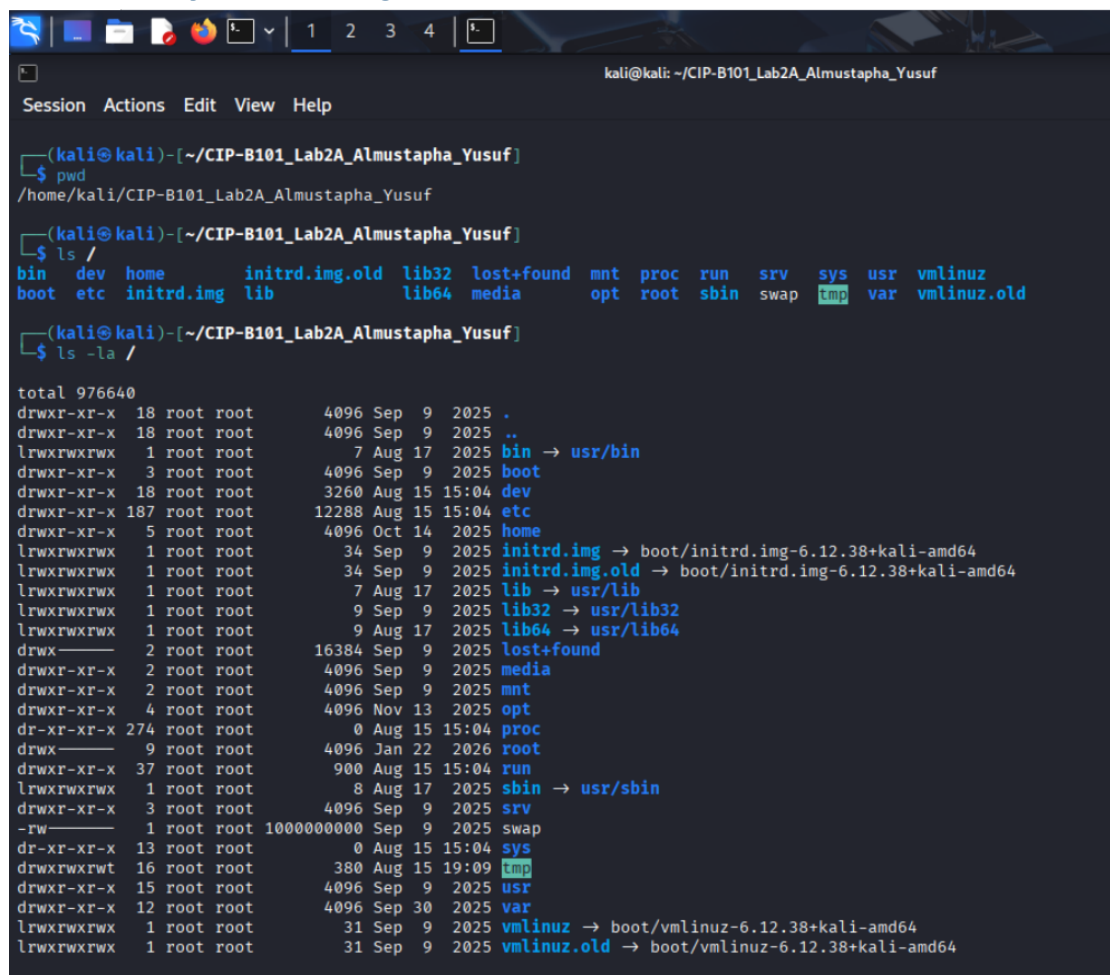
Screenshot 1: Evidence workspace directory structure created and verified with `ls -la`

The `mkdir -p` command created the parent directory and all six subdirectories in a single operation. The `-p` flag ensures that no errors are raised if the directory already exists and that any necessary parent directories are created automatically. The `pwd` command confirmed the working directory as `/home/kali/CIP-B101_Lab2A_Almustapha_Yusuf`, and `ls -la` verified that all six subdirectories (archive, evidence, notes, outputs, screenshots, scripts) were created with permissions `drwxrwxr-x` (775).

3. Task 2: Linux Filesystem and OS Information

Understanding the Linux filesystem hierarchy is fundamental for forensic work because evidence can reside in many different locations across the system. The root directory (`/`) serves as the top-level entry point, with each subdirectory serving a specific purpose. In this task, several commands were used to inspect the operating system configuration and user context, which is a standard first step in any forensic investigation to establish the baseline environment of the system under examination.

3.1 Root Filesystem Listing



```
(kali@kali)~[/CIP-B101_Lab2A_Almustapha_Yusuf]
$ pwd
/home/kali/CIP-B101_Lab2A_Almustapha_Yusuf

(kali@kali)~[/CIP-B101_Lab2A_Almustapha_Yusuf]
$ ls /
bin  dev  home  initrd.img.old  lib32  lost+found  mnt  proc  run  srv  sys  usr  vmlinuz
boot  etc  initrd.img  lib  lib64  media  opt  root  sbin  swap  tmp  var  vmlinuz.old

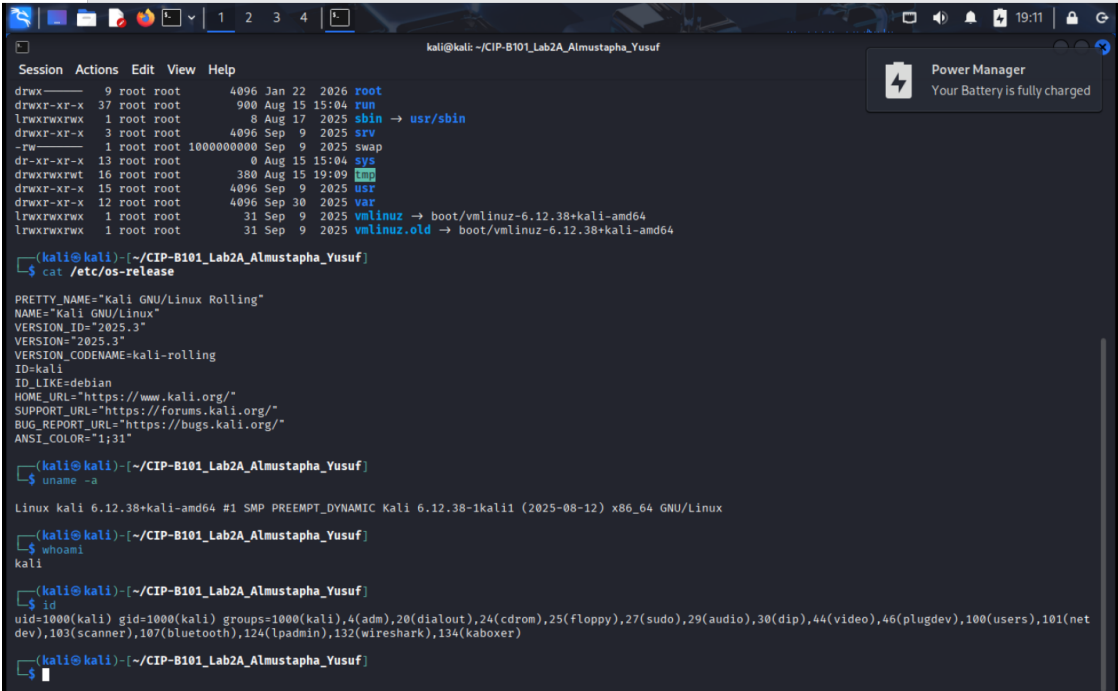
(kali@kali)~[/CIP-B101_Lab2A_Almustapha_Yusuf]
$ ls -la /
total 976640
drwxr-xr-x 18 root root 4096 Sep  9  2025 .
drwxr-xr-x 18 root root 4096 Sep  9  2025 ..
lrwxrwxrwx  1 root root    7 Aug 17  2025 bin -> usr/bin
drwxr-xr-x  3 root root 4096 Sep  9  2025 boot
drwxr-xr-x 18 root root 3260 Aug 15 15:04 dev
drwxr-xr-x 187 root root 12288 Aug 15 15:04 etc
drwxr-xr-x  5 root root 4096 Oct 14  2025 home
lrwxrwxrwx  1 root root   34 Sep  9  2025 initrd.img -> boot/initrd.img-6.12.38+kali-amd64
lrwxrwxrwx  1 root root   34 Sep  9  2025 initrd.img.old -> boot/initrd.img-6.12.38+kali-amd64
lrwxrwxrwx  1 root root    7 Aug 17  2025 lib -> usr/lib
lrwxrwxrwx  1 root root    9 Sep  9  2025 lib32 -> usr/lib32
lrwxrwxrwx  1 root root    9 Aug 17  2025 lib64 -> usr/lib64
drwx-----  2 root root 16384 Sep  9  2025 lost+found
drwxr-xr-x  2 root root 4096 Sep  9  2025 media
drwxr-xr-x  2 root root 4096 Sep  9  2025 mnt
drwxr-xr-x  4 root root 4096 Nov 13  2025 opt
dr-xr-xr-x 274 root root    0 Aug 15 15:04 proc
drwx-----  9 root root 4096 Jan 22  2026 root
drwxr-xr-x 37 root root 900 Aug 15 15:04 run
lrwxrwxrwx  1 root root    8 Aug 17  2025 sbin -> usr/sbin
drwxr-xr-x  3 root root 4096 Sep  9  2025 srv
-rw-----  1 root root 1000000000 Sep  9  2025 swap
dr-xr-xr-x 13 root root    0 Aug 15 15:04 sys
drwxrwxrwt 16 root root 380 Aug 15 19:09 tmp
drwxr-xr-x 15 root root 4096 Sep  9  2025 usr
drwxr-xr-x 12 root root 4096 Sep 30  2025 var
lrwxrwxrwx  1 root root   31 Sep  9  2025 vmlinuz -> boot/vmlinuz-6.12.38+kali-amd64
lrwxrwxrwx  1 root root   31 Sep  9  2025 vmlinuz.old -> boot/vmlinuz-6.12.38+kali-amd64
```

Screenshot 2.1: Root filesystem listing with `ls /` and `ls -la /`

The root directory listing revealed the standard Linux filesystem hierarchy including `bin` (user binaries, symlinked to `usr/bin`), `boot` (kernel and bootloader files), `dev` (device files), `etc` (configuration files), `home` (user directories), `lib` and `lib64` (system libraries), `proc` (process and kernel information), `sys` (system hardware information), `tmp` (temporary files), and `var` (variable data).

such as logs). The ls -la output showed symbolic links for bin, lib, lib32, lib64, and sbin pointing to their corresponding locations under /usr, which is a characteristic of modern systemd-based Linux distributions.

3.2 Operating System and User Information



```
kali@kali: ~/CIP-B101_Lab2A_Almustapha_Yusuf
Session Actions Edit View Help
drwx----- 9 root root 4096 Jan 22 2026 root
drwxr-xr-x 37 root root 900 Aug 15 15:04 run
lrwxrwxrwx 1 root root 8 Aug 17 2025 sbin -> usr/sbin
drwxr-xr-x 3 root root 4096 Sep 9 2025 srv
-rw----- 1 root root 1000000000 Sep 9 2025 swap
dr-xr-xr-x 13 root root 0 Aug 15 15:04 sys
drwxrwxrwt 16 root root 380 Aug 15 19:09 tmp
drwxr-xr-x 15 root root 4096 Sep 9 2025 usr
drwxr-xr-x 12 root root 4096 Sep 30 2025 var
lrwxrwxrwx 1 root root 31 Sep 9 2025 vmlinuz -> boot/vmlinuz-6.12.38+kali-amd64
lrwxrwxrwx 1 root root 31 Sep 9 2025 vmlinuz.old -> boot/vmlinuz-6.12.38+kali-amd64

(kali@kali)~[/CIP-B101_Lab2A_Almustapha_Yusuf]
$ cat /etc/os-release
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2025.3"
VERSION="2025.3"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
HOME_URL="https://www.kali.org/"
SUPPORT_URL="https://forums.kali.org/"
BUG_REPORT_URL="https://bugs.kali.org/"
ANSI_COLOR="1;31"

(kali@kali)~[/CIP-B101_Lab2A_Almustapha_Yusuf]
$ uname -a
Linux kali 6.12.38+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.38-1kali1 (2025-08-12) x86_64 GNU/Linux

(kali@kali)~[/CIP-B101_Lab2A_Almustapha_Yusuf]
$ whoami
kali

(kali@kali)~[/CIP-B101_Lab2A_Almustapha_Yusuf]
$ id
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(netdev),103(scanner),107(bluetooth),124(lpadmin),132(wireshark),134(kaboxer)

(kali@kali)~[/CIP-B101_Lab2A_Almustapha_Yusuf]
$
```

Screenshot 2.2: OS release, kernel, user identity, and group membership

Property	Value
Pretty_Name	Kali GNU/Linux Rolling
Version_Id	2025.3
Kernel	6.12.38+kali-amd64
Architecture	x86_64 GNU/Linux
User	kali (uid=1000)
Groups	kali, adm, dialout, cdrom, sudo, audio, dip, video, plugdev, users, netdev, scanner, bluetooth, lpadmin, wireshark, kaboxer

Table 1: System Information Summary

The cat /etc/os-release command revealed Kali Linux 2025.3 Rolling release, which is the latest version of the penetration testing and forensics distribution. The uname -a output confirmed the kernel version as 6.12.38+kali-amd64 with SMP PREEMPT_DYNAMIC support, running on x86_64 architecture. The whoami and id commands confirmed the current user is kali (uid=1000) with membership in several important groups including sudo (administrative privileges), wireshark (network capture permissions), and netdev (network interface configuration), all of which are relevant to forensic operations.

4. Task 3: Inodes and Disk Information

An inode (index node) is a fundamental data structure in Unix/Linux file systems that stores metadata about a file, including its size, permissions, ownership, timestamps, and the locations of

its data blocks on disk. Crucially, the inode does not store the file name itself - that is stored in the directory entry that points to the inode. This separation of file names from file metadata is extremely important in digital forensics because it means that even if a file name is changed or a file is deleted (which removes the directory entry but not necessarily the inode), the file data and much of its metadata may still be recoverable. Forensic tools like The Sleuth Kit and Autopsy work directly with inodes to recover deleted files and analyze file system activity.

```
echo "This is Lab 2A evidence file created by Almustapha Yusuf" >
evidence/lab2a_evidence.txt

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ echo "This is Lab 2A evidence file created by Almustapha Yusuf" > evidence/lab2a_evidence.txt

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ ls -li evidence/lab2a_evidence.txt

3545830 -rw-rw-r-- 1 kali kali 57 Aug 15 19:15 evidence/lab2a_evidence.txt

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ stat evidence/lab2a_evidence.txt

File: evidence/lab2a_evidence.txt
Size: 57          Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 3545830    Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-08-15 19:15:26.931919089 -0400
Modify: 2026-08-15 19:15:26.931919089 -0400
Change: 2026-08-15 19:15:26.931919089 -0400
Birth: 2026-08-15 19:15:26.931919089 -0400

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ df -h

Filesystem      Size  Used Avail Use% Mounted on
udev            916M   0    916M   0% /dev
tmpfs           197M  1.3M  196M   1% /run
/dev/sda1       79G   26G   49G   35% /
tmpfs           983M  4.0K  983M   1% /dev/shm
tmpfs           5.0M   0    5.0M   0% /run/lock
tmpfs           1.0M   0    1.0M   0% /run/credentials/systemd-journald.service
tmpfs           983M  64K  983M   1% /tmp
tmpfs           1.0M   0    1.0M   0% /run/credentials/getty@tty1.service
tmpfs           197M  124K  197M   1% /run/user/1000

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ df -i

Filesystem      Inodes   IUsed   IFree IUse% Mounted on
udev            234454    402   234052    1% /dev
tmpfs           251457    721   250736    1% /run
/dev/sda1       5251072 515448 4735624   10% /
tmpfs           251457     2    251455    1% /dev/shm
```

Screenshot 3.1: Evidence file creation, inode inspection, and disk information

The `ls -li` command displayed the inode number (3545830) alongside the file listing, confirming that each file in the filesystem is tracked by a unique inode identifier. The `stat` command provided comprehensive metadata including the file size (57 bytes), block count (8 blocks of 4096 bytes each), device identifier (8,1 representing `/dev/sda1`), permissions (0664), and all four timestamps: access, modify, change, and birth. The `df -h` command showed the primary partition `/dev/sda1` with 79G total capacity, 26G used (35% utilization), and 49G available. The `df -i` command revealed inode usage at 10% (515,448 of 5,251,072 inodes used), which is an important metric because even if disk space is available, a filesystem can run out of inodes and be unable to create new files.

```
(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ stat evidence/lab2a_evidence.txt

File: evidence/lab2a_evidence.txt
Size: 57          Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 3545830   Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-08-15 19:15:26.931919089 -0400
Modify: 2026-08-15 19:15:26.931919089 -0400
Change: 2026-08-15 19:15:26.931919089 -0400
Birth: 2026-08-15 19:15:26.931919089 -0400

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ df -h

Filesystem      Size  Used Avail Use% Mounted on
udev            916M   0  916M   0% /dev
tmpfs           197M  1.3M  196M   1% /run
/dev/sda1       79G   26G   49G  35% /
tmpfs           983M  4.0K  983M   1% /dev/shm
tmpfs           5.0M   0   5.0M   0% /run/lock
tmpfs           1.0M   0   1.0M   0% /run/credentials/systemd-journald.service
tmpfs           983M  64K  983M   1% /tmp
tmpfs           1.0M   0   1.0M   0% /run/credentials/getty@tty1.service
tmpfs          197M  124K  197M   1% /run/user/1000

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ df -i

Filesystem      Inodes   IUsed   IFree IUse% Mounted on
udev            234454    402  234052    1% /dev
tmpfs           251457    721  250736    1% /run
/dev/sda1       5251072 515448 4735624   10% /
tmpfs           251457     2  251455    1% /dev/shm
tmpfs           251457     2  251455    1% /run/lock
tmpfs           1024      1   1023    1% /run/credentials/systemd-journald.service
tmpfs          1048576    31 1048545    1% /tmp
tmpfs           1024      1   1023    1% /run/credentials/getty@tty1.service
tmpfs          50291    132  50159    1% /run/user/1000

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$
```

Screenshot 3.2: Full stat, df -h, and df -i output (continued)

5. Task 4: Absolute and Relative Paths

Understanding the difference between absolute and relative paths is essential for forensic work because evidence files must be referenced precisely and unambiguously in reports and chain-of-custody documentation. An absolute path specifies the complete path from the root directory (/) to the target file, beginning with a forward slash. A relative path specifies the location of a file relative to the current working directory, using dot (.) for the current directory and double dot (..) for the parent directory. In forensic reporting, absolute paths are generally preferred because they eliminate any ambiguity about the exact location of evidence files, regardless of which directory the investigator is currently in.


```
(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ cd ~/CIP-B101_Lab2A_Almustapha_Yusuf/evidence

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/evidence]
$ pwd
/home/kali/CIP-B101_Lab2A_Almustapha_Yusuf/evidence

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/evidence]
$ cd ..

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ pwd
/home/kali/CIP-B101_Lab2A_Almustapha_Yusuf

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ cd ./notes

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/notes]
$ pwd
/home/kali/CIP-B101_Lab2A_Almustapha_Yusuf/notes

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/notes]
$ cd ../evidence

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/evidence]
$ pwd
/home/kali/CIP-B101_Lab2A_Almustapha_Yusuf/evidence

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/evidence]
$ readlink -f lab2a_evidence.txt
/home/kali/CIP-B101_Lab2A_Almustapha_Yusuf/evidence/lab2a_evidence.txt

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/evidence]
$
```

Screenshot 4: Absolute and relative path navigation demonstration

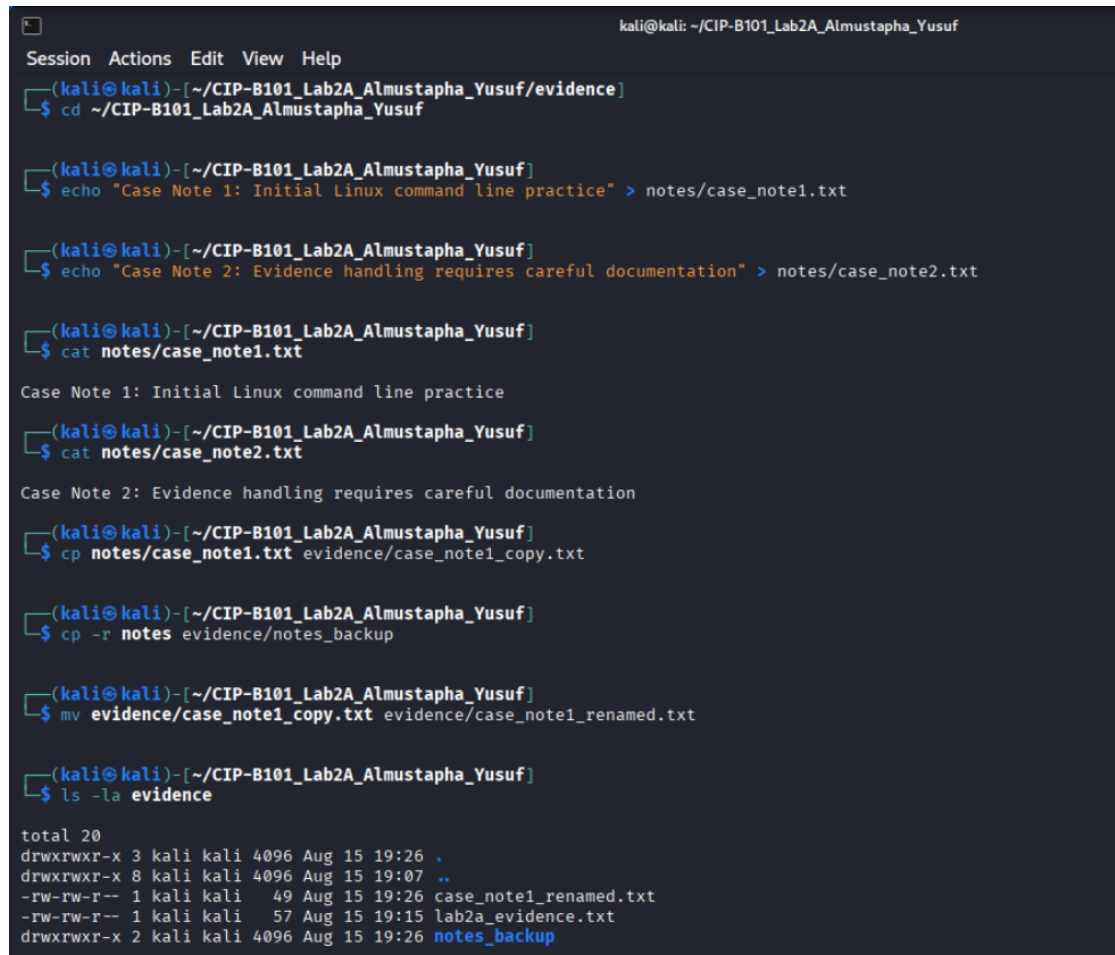
The navigation sequence demonstrated both absolute and relative path usage. The `cd` command with an absolute path (`~/CIP-B101_Lab2A_Almustapha_Yusuf/evidence`) moved directly to the evidence directory regardless of the current location. The relative path `cd ..` moved up one level to the parent directory, and `cd ./notes` moved into the notes subdirectory using the dot notation. The `readlink -f` command resolved the relative path `lab2a_evidence.txt` to its full absolute path (`/home/kali/CIP-B101_Lab2A_Almustapha_Yusuf/evidence/lab2a_evidence.txt`), which is the form used in forensic documentation to unambiguously identify the evidence file location.

Path Type	Example to lab2a_evidence.txt
Absolute path	/home/kali/CIP-B101_Lab2A_Almustapha_Yusuf/evidence/lab2a_evidence.txt
Relative path (from evidence/)	lab2a_evidence.txt
Relative path (from parent)	evidence/lab2a_evidence.txt

Table 2: Absolute vs Relative Path Examples

6. Task 5: File Management Operations

File management operations such as create, copy, rename, and delete are among the most frequently used commands in digital forensics. Investigators regularly need to create working copies of evidence files (to preserve originals), organize extracted data into structured directories, rename files to follow case naming conventions, and clean up temporary analysis files. The `cp` command creates duplicates while preserving the original, the `mv` command renames or relocates files, and the `rm` command permanently deletes files. In forensic contexts, extra care must be taken with `rm` because deleted files may not be recoverable on all filesystems, and the original evidence must never be modified or deleted.

A terminal window titled 'kali@kali: ~/CIP-B101_Lab2A_Almustapha_Yusuf' showing a series of commands and their outputs. The user navigates to the 'evidence' directory, creates two text files using 'echo', copies one file to the 'evidence' directory, recursively copies the 'notes' directory to 'evidence/notes_backup', renames the copied file, and finally lists the contents of the 'evidence' directory. The 'ls -la' command shows the directory structure with permissions, owner, group, size, and timestamps for the files and subdirectories.

```
kali@kali: ~/CIP-B101_Lab2A_Almustapha_Yusuf
Session Actions Edit View Help

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/evidence]
$ cd ~/CIP-B101_Lab2A_Almustapha_Yusuf

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ echo "Case Note 1: Initial Linux command line practice" > notes/case_note1.txt

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ echo "Case Note 2: Evidence handling requires careful documentation" > notes/case_note2.txt

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ cat notes/case_note1.txt

Case Note 1: Initial Linux command line practice

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ cat notes/case_note2.txt

Case Note 2: Evidence handling requires careful documentation

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ cp notes/case_note1.txt evidence/case_note1_copy.txt

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ cp -r notes evidence/notes_backup

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ mv evidence/case_note1_copy.txt evidence/case_note1_renamed.txt

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ ls -la evidence

total 20
drwxrwxr-x 3 kali kali 4096 Aug 15 19:26 .
drwxrwxr-x 8 kali kali 4096 Aug 15 19:07 ..
-rw-rw-r-- 1 kali kali  49 Aug 15 19:26 case_note1_renamed.txt
-rw-rw-r-- 1 kali kali  57 Aug 15 19:15 lab2a_evidence.txt
drwxrwxr-x 2 kali kali 4096 Aug 15 19:26 notes_backup
```

Screenshot 5: File creation, copy, rename, and delete operations

Two case note files were created using `echo` with output redirection. The `cp` command copied `case_note1.txt` to the evidence directory, and `cp -r` recursively copied the entire notes directory to `evidence/notes_backup`. The `mv` command renamed `case_note1_copy.txt` to `case_note1_renamed.txt` within the evidence directory. A temporary file was created and then removed with `rm` to demonstrate the deletion operation. The final `ls -la` confirmed the evidence directory contains the original `lab2a_evidence.txt`, the renamed `case_note1_renamed.txt`, and the `notes_backup` subdirectory with both case notes preserved inside.

7. Task 6: Searching Evidence Content with grep

The `grep` command is one of the most powerful and frequently used tools in digital forensics for searching through log files, evidence text, and configuration files. It searches for patterns (including regular expressions) within files and returns matching lines. In forensic investigations, `grep` is used to quickly identify relevant entries in large log files, find indicators of compromise, extract IP

addresses or usernames from evidence, and filter data for further analysis. The various grep flags used in this task demonstrate different search capabilities: basic pattern matching (-n for line numbers, -E for extended regex, -i for case insensitivity) that are essential for flexible evidence searching.

```
(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ cat > evidence/activity_log.txt <<'EOF'

heredoc> 2026-06-19 09:10 user=aminu action=login status=success ip=192.168.56.10
2026-06-19 09:25 user=guest action=file_copy status=success file=report.docx
2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15
2026-06-19 10:20 user=aminu action=delete status=success file=temp.txt
2026-06-19 10:40 user=admin action=password_change status=success
EOF

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ cat evidence/activity_log.txt

2026-06-19 09:10 user=aminu action=login status=success ip=192.168.56.10
2026-06-19 09:25 user=guest action=file_copy status=success file=report.docx
2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15
2026-06-19 10:20 user=aminu action=delete status=success file=temp.txt
2026-06-19 10:40 user=admin action=password_change status=success

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ grep "login" evidence/activity_log.txt

2026-06-19 09:10 user=aminu action=login status=success ip=192.168.56.10
2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ grep -n "failed" evidence/activity_log.txt

4:2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ grep -Ei "delete|password" evidence/activity_log.txt

2026-06-19 10:20 user=aminu action=delete status=success file=temp.txt
2026-06-19 10:40 user=admin action=password_change status=success

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ grep -Ei "192\.168\.56\. [0-9]+" evidence/activity_log.txt
2026-06-19 09:10 user=aminu action=login status=success ip=192.168.56.10
2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
```

Screenshot 6: Activity log creation and grep search operations

Command	Results	Description
grep "login"	2 matches	Login events (aminu success, admin failed)
grep -n "failed"	1 match (line 4)	Failed login by admin from 192.168.56.15
grep -Ei "delete password"	2 matches	Delete action + password change by admin
grep -Ei "192\.168\.56\. [0-9]+"	2 matches	IP addresses from log entries

Table 3: grep Search Results Summary

The grep 'login' command identified two login events: a successful login by user aminu from IP 192.168.56.10 and a failed login attempt by admin from 192.168.56.15. The grep -n 'failed' command precisely located the failed login on line 4 of the activity log, which would be the primary evidence of a potential unauthorized access attempt. The grep -Ei 'delete|password' command used extended regex with case insensitivity to find both a file deletion by aminu and a password change by admin, which together could indicate suspicious account activity following a failed login. The IP address pattern search extracted both log entries containing 192.168.56.x addresses, demonstrating how regex patterns can target specific data formats in evidence files.

8. Task 7: Directory Tree and File Discovery

The `tree` and `find` commands are essential for mapping the structure of evidence directories and locating specific files across complex directory hierarchies. In forensic investigations, understanding the complete directory structure of a suspect system or evidence workspace helps investigators identify where relevant files might be stored. The `tree` command provides a visual representation of the directory hierarchy, while `find` offers powerful search capabilities with filters for file type, name patterns (including wildcards and case-insensitive matching), size, date, permissions, and many other criteria. Forensic investigators frequently use `find` to locate all files modified within a specific time window or all files of a particular type across an entire filesystem.

```
Session Actions Edit View Help
(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ cd ~/CIP-B101_Lab2A_Almustapha_Yusuf

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ tree
.
├── archive
├── evidence
│   ├── activity_log.txt
│   ├── case_note1_renamed.txt
│   ├── lab2a_evidence.txt
│   └── notes_backup
│       ├── case_note1.txt
│       └── case_note2.txt
├── notes
│   ├── case_note1.txt
│   └── case_note2.txt
├── outputs
├── screenshots
└── scripts

8 directories, 7 files

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ find . -type f -name "*.txt"

./evidence/lab2a_evidence.txt
./evidence/activity_log.txt
./evidence/notes_backup/case_note2.txt
./evidence/notes_backup/case_note1.txt
./evidence/case_note1_renamed.txt
./notes/case_note2.txt
./notes/case_note1.txt

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ find . -type f -iname "*log*"

./evidence/activity_log.txt

(kali㉿kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ find . -type d -name "evidence"

./evidence
```

Screenshot 7: `tree` output and `find` command results

The `tree` command displayed the complete workspace structure: 8 directories and 7 files. The `find` commands demonstrated three different search patterns. The `find . -type f -name '*.txt'` located all 7 text files across all subdirectories. The `find . -type f -iname '*log*'` used case-insensitive name matching to find the single `activity_log.txt` file. The `find . -type d -name 'evidence'` searched specifically for directories named 'evidence', finding the one at `./evidence`. These search patterns demonstrate how `find` can be used to systematically locate evidence files in a forensic investigation.

9. Task 8: Network Information (Localhost Only)

Checking network configuration is a standard step in forensic investigations to understand the system's network context, identify active interfaces, and determine what network services are running. This lab specifically restricts network testing to localhost (127.0.0.1) to emphasize safe practices - forensic trainees must never scan or probe unauthorized external systems. Localhost testing is safe because it only involves the local machine's loopback interface, which cannot affect other systems. Understanding why this restriction exists is itself an important lesson: unauthorized network scanning is illegal in many jurisdictions and can be interpreted as a hostile action.

```
Session Actions Edit View Help
(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ ip addr

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:f3:f7:44 brd ff:ff:ff:ff:ff:ff
    inet 192.168.199.128/24 brd 192.168.199.255 scope global dynamic noprefixroute eth0
        valid_lft 1546sec preferred_lft 1546sec
    inet6 fe80::8c84:5c9c:8758:3178/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ ip route

default via 192.168.199.2 dev eth0 proto dhcp src 192.168.199.128 metric 100
192.168.199.0/24 dev eth0 proto kernel scope link src 192.168.199.128 metric 100

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ ping -c 4 127.0.0.1

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=2.18 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.027 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.026 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.023 ms

— 127.0.0.1 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3037ms
rtt min/avg/max/mdev = 0.023/0.563/2.177/0.931 ms

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ ping -c 4 localhost

PING localhost (::1) 56 data bytes
64 bytes from localhost (::1): icmp_seq=1 ttl=64 time=1.51 ms
64 bytes from localhost (::1): icmp_seq=2 ttl=64 time=0.028 ms
64 bytes from localhost (::1): icmp_seq=3 ttl=64 time=0.054 ms
64 bytes from localhost (::1): icmp_seq=4 ttl=64 time=0.034 ms
```

Screenshot 8.1: Network interfaces, routing, and loopback ping tests

The ip addr command showed two interfaces: the loopback interface (lo) with IP 127.0.0.1/8 and the Ethernet interface (eth0) with IP 192.168.199.128/24. The ip route command displayed the default gateway at 192.168.199.2 via eth0. Both ping tests to 127.0.0.1 (IPv4) and localhost (::1, IPv6) completed successfully with 0% packet loss, confirming that the network stack is functioning correctly. The loopback address 127.0.0.1 is a special address that routes traffic back to the local machine without ever reaching the physical network, making it completely safe for testing purposes.

```
Session Actions Edit View Help
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.026 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.023 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3037ms
rtt min/avg/max/mdev = 0.023/0.563/2.177/0.931 ms

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ ping -c 4 localhost

PING localhost (::1) 56 data bytes
64 bytes from localhost (::1): icmp_seq=1 ttl=64 time=1.51 ms
64 bytes from localhost (::1): icmp_seq=2 ttl=64 time=0.028 ms
64 bytes from localhost (::1): icmp_seq=3 ttl=64 time=0.054 ms
64 bytes from localhost (::1): icmp_seq=4 ttl=64 time=0.034 ms

--- localhost ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 0.028/0.406/1.508/0.636 ms

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ ss -ltn

State          Recv-Q         Send-Q         Local Address:Port      Peer Address:Port
LISTEN         0               128            0.0.0.0:22              0.0.0.0:*
LISTEN         0               128            [::]:22                  [::]:*

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ nmap localhost -p 22,80,443 2>/dev/null || echo "nmap unavailable or no services detected"
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-15 19:39 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00031s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    closed http
443/tcp   closed https

Nmap done: 1 IP address (1 host up) scanned in 0.14 seconds

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
```

Screenshot 8.2: Listening ports (ss) and Nmap localhost scan

Port	Service	State	Notes
22/tcp	ssh	open	SSH daemon (sshd) listening on IPv4 and IPv6
80/tcp	http	closed	No web server running
443/tcp	https	closed	No HTTPS server running

Table 4: Nmap Localhost Port Scan Results

The `ss -ltn` command confirmed that only SSH (port 22) is listening, on both IPv4 (0.0.0.0:22) and IPv6 (:::22). The Nmap scan of localhost on ports 22, 80, and 443 corroborated this finding: port 22 is open for SSH, while ports 80 (HTTP) and 443 (HTTPS) are closed. This minimal service footprint is typical for a Kali Linux system that is not running any web services, and it reduces the attack surface of the forensic workstation.

10. Task 9: Forensic Shell Script

Shell scripting is a critical skill for digital forensic professionals because it enables automation of repetitive tasks, standardization of evidence collection procedures, and creation of reusable tools for common forensic operations. A well-written forensic script can collect system information, capture network state, document running processes, and save all results to a structured output file in a single execution. This ensures consistency across multiple examinations and reduces the risk of human error during manual command execution. In professional forensic labs, scripts are often used for first-response triage to quickly capture volatile data before it is lost or altered.

```
Session Actions Edit View Help
(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf]
$ cd ~/CIP-B101_Lab2A_Almustapha_Yusuf/scripts

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/scripts]
$ nano lab2a_sys_summary.sh

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/scripts]
$ chmod +x lab2a_sys_summary.sh

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/scripts]
$ ls -l lab2a_sys_summary.sh

-rwxrwxr-x 1 kali kali 563 Aug 15 19:52 lab2a_sys_summary.sh

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/scripts]
$ ./lab2a_sys_summary.sh

Summary saved to /home/kali/CIP-B101_Lab2A_Almustapha_Yusuf/outputs/system_summary.txt

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/scripts]
$ cat ../outputs/system_summary.txt | head -n 30
===== CIP-B101 Lab 2A System Summary =====
Date/Time: Sat Aug 15 07:54:32 PM EDT 2026
User: kali
Hostname: kali
Kernel: Linux kali 6.12.38+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.38-1kali1 (2025-08-12) x86_64 GNU/Linux
Current Folder: /home/kali/CIP-B101_Lab2A_Almustapha_Yusuf/scripts
===== Disk Space =====
Filesystem      Size  Used Avail Use% Mounted on
udev             916M   0  916M   0% /dev
tmpfs            197M  1.3M  196M   1% /run
/dev/sda1        79G   26G   49G  35% /
tmpfs            983M  4.0K  983M   1% /dev/shm
tmpfs            5.0M   0   5.0M   0% /run/lock
tmpfs            1.0M   0   1.0M   0% /run/credentials/systemd-journald.service
tmpfs            983M  376K  982M   1% /tmp
tmpfs            1.0M   0   1.0M   0% /run/credentials/getty@tty1.service
tmpfs            197M  124K  197M   1% /run/user/1000
===== Inode Usage =====
Filesystem      Inodes  IUsed  IFree IUse% Mounted on
udev            234454   402  234052   1% /dev
```

Screenshot 9.1: Script creation, permission change, execution, and output verification

The lab2a_sys_summary.sh script was created in nano with a shebang line (#!/bin/sh) specifying the shell interpreter. The script captures the date/time, current user, hostname, kernel information, current working directory, disk space usage (df -h), inode usage (df -i), and the evidence workspace tree structure. All output is redirected to a single file using command grouping with curly braces and output redirection. The chmod +x command added execute permission, changing the file mode from -rw-rw-r-- to -rwxrwxr-x. The script executed successfully and produced the system_summary.txt file in the outputs directory.


```
Session Actions Edit View Help

-rwxrwxr-x 1 kali kali 563 Aug 15 19:52 lab2a_sys_summary.sh

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/scripts]
$ ./lab2a_sys_summary.sh

Summary saved to /home/kali/CIP-B101_Lab2A_Almustapha_Yusuf/outputs/system_summary.txt

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/scripts]
$ cat ../outputs/system_summary.txt | head -n 30
===== CIP-B101 Lab 2A System Summary =====
Date/Time: Sat Aug 15 07:54:32 PM EDT 2026
User: kali
Hostname: kali
Kernel: Linux kali 6.12.38+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.38-1kali1 (2025-08-12) x86_64 GNU/Linux
Current Folder: /home/kali/CIP-B101_Lab2A_Almustapha_Yusuf/scripts
===== Disk Space =====
Filesystem      Size  Used Avail Use% Mounted on
udev            916M   0  916M   0% /dev
tmpfs           197M  1.3M  196M   1% /run
/dev/sda1       79G   26G   49G  35% /
tmpfs           983M  4.0K  983M   1% /dev/shm
tmpfs           5.0M   0   5.0M   0% /run/lock
tmpfs           1.0M   0   1.0M   0% /run/credentials/systemd-journald.service
tmpfs           983M  376K  982M   1% /tmp
tmpfs           1.0M   0   1.0M   0% /run/credentials/getty@tty1.service
tmpfs          197M  124K  197M   1% /run/user/1000
===== Inode Usage =====
Filesystem      Inodes  IUsed  IFree IUse% Mounted on
udev            234454   402  234052    1% /dev
tmpfs           251457   722  250735    1% /run
/dev/sda1       5251072 515457 4735615   10% /
tmpfs           251457     2  251455    1% /dev/shm
tmpfs           251457     2  251455    1% /run/lock
tmpfs           1024     1    1023    1% /run/credentials/systemd-journald.service
tmpfs          1048576    31 1048545    1% /tmp
tmpfs           1024     1    1023    1% /run/credentials/getty@tty1.service
tmpfs           50291    133  50158    1% /run/user/1000
===== Evidence Workspace Tree =====
/home/kali/CIP-B101_Lab2A_Almustapha_Yusuf

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/scripts]
$
```

Screenshot 9.2: System summary output showing disk, inode, and workspace tree data

11. Task 10: PATH Variable and Script Accessibility

The PATH environment variable is a colon-separated list of directories that the shell searches when a command is typed without specifying its full path. When you type 'ls', the shell searches each directory in PATH until it finds the ls executable. This mechanism allows commands like ls, cd, and grep to run from any directory without typing their full path (e.g., /usr/bin/ls). By adding a custom scripts directory to PATH, any executable script placed in that directory becomes accessible from anywhere on the system, just like built-in commands. In forensic work, this is useful for creating a toolkit of custom scripts that can be invoked quickly during evidence collection without remembering complex paths.


```
Session Actions Edit View Help
(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/scripts]
$ export PATH=$HOME/CIP-B101_Lab2A_Almustapha_Yusuf/scripts:$PATH

(kali@kali)-[~/CIP-B101_Lab2A_Almustapha_Yusuf/scripts]
$ cd /tmp

(kali@kali)-[/tmp]
$ lab2a_sys_summary.sh

Summary saved to /home/kali/CIP-B101_Lab2A_Almustapha_Yusuf/outputs/system_summary.txt

(kali@kali)-[/tmp]
$ cat ~/CIP-B101_Lab2A_Almustapha_Yusuf/outputs/system_summary.txt | head -n 15
===== CIP-B101 Lab 2A System Summary =====
Date/Time: Sat Aug 15 07:57:35 PM EDT 2026
User: kali
Hostname: kali
Kernel: Linux kali 6.12.38+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.38-1kali1 (2025-08-12) x86_64 GNU/Linux
Current Folder: /tmp
===== Disk Space =====
Filesystem      Size  Used Avail Use% Mounted on
udev            916M   0  916M   0% /dev
tmpfs           197M  1.3M  196M   1% /run
/dev/sda1        79G   26G   49G  35% /
tmpfs           983M  4.0K  983M   1% /dev/shm
tmpfs           5.0M   0   5.0M   0% /run/lock
tmpfs           1.0M   0   1.0M   0% /run/credentials/systemd-journald.service
tmpfs           983M  424K  982M   1% /tmp

(kali@kali)-[/tmp]
$
```

Screenshot 10: Script executed from /tmp after adding scripts directory to PATH

The `export PATH=$HOME/CIP-B101_Lab2A_Almustapha_Yusuf/scripts:$PATH` command prepended the scripts directory to the beginning of PATH. After changing to /tmp with `cd /tmp`, the script `lab2a_sys_summary.sh` was executed by name alone, without specifying its full path. The script ran successfully, and the output file confirmed 'Current Folder: /tmp', proving that the script was indeed executed from /tmp and not from the scripts directory. This demonstrates how PATH enables command accessibility across different working directories, which is the same reason standard Linux commands like `ls` and `grep` work regardless of which directory the user is currently in.

12. Task 11: Evidence Archiving

Compressing and archiving evidence is the final step in the evidence handling workflow, creating a single portable package that preserves the original directory structure, file permissions, and timestamps. Both `zip` and `tar.gz` are widely used formats in forensic work. The `zip` format is universally supported across operating systems (Windows, macOS, Linux) and uses the ZIP compression algorithm. The `tar.gz` format combines `tar` (which bundles files into a single archive preserving metadata) with `gzip` compression (which reduces the archive size). The `tar` format is native to Unix/Linux systems and better preserves file ownership and permissions, making it the preferred format for Linux-based evidence archival.

```
Session Actions Edit View Help
(kali@kali)-[/tmp]
$ cd ~

(kali@kali)-[~]
$ zip -r CIP-B101_Lab2A_Almustapha_Yusuf.zip CIP-B101_Lab2A_Almustapha_Yusuf

adding: CIP-B101_Lab2A_Almustapha_Yusuf/ (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/archive/ (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/evidence/ (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/evidence/lab2a_evidence.txt (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/evidence/activity_log.txt (deflated 55%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/evidence/notes_backup/ (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/evidence/notes_backup/case_note2.txt (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/evidence/notes_backup/case_note1.txt (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/evidence/case_note1_renamed.txt (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/notes/ (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/notes/case_note2.txt (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/notes/case_note1.txt (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/outputs/ (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/outputs/system_summary.txt (deflated 61%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/screenshots/ (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/scripts/ (stored 0%)
adding: CIP-B101_Lab2A_Almustapha_Yusuf/scripts/lab2a_sys_summary.sh (deflated 48%)

(kali@kali)-[~]
$ tar -czvf CIP-B101_Lab2A_Almustapha_Yusuf.tar.gz CIP-B101_Lab2A_Almustapha_Yusuf

CIP-B101_Lab2A_Almustapha_Yusuf/
CIP-B101_Lab2A_Almustapha_Yusuf/archive/
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/lab2a_evidence.txt
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/activity_log.txt
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/notes_backup/
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/notes_backup/case_note2.txt
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/notes_backup/case_note1.txt
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/case_note1_renamed.txt
CIP-B101_Lab2A_Almustapha_Yusuf/notes/
CIP-B101_Lab2A_Almustapha_Yusuf/notes/case_note2.txt
CIP-B101_Lab2A_Almustapha_Yusuf/notes/case_note1.txt
CIP-B101_Lab2A_Almustapha_Yusuf/outputs/
CIP-B101_Lab2A_Almustapha_Yusuf/outputs/system_summary.txt
CIP-B101_Lab2A_Almustapha_Yusuf/screenshots/
```

Screenshot 11.1: zip and tar.gz archive creation with verbose output

```
(kali@kali)-[~]
$ tar -czvf CIP-B101_Lab2A_Almustapha_Yusuf.tar.gz CIP-B101_Lab2A_Almustapha_Yusuf

CIP-B101_Lab2A_Almustapha_Yusuf/
CIP-B101_Lab2A_Almustapha_Yusuf/archive/
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/lab2a_evidence.txt
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/activity_log.txt
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/notes_backup/
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/notes_backup/case_note2.txt
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/notes_backup/case_note1.txt
CIP-B101_Lab2A_Almustapha_Yusuf/evidence/case_note1_renamed.txt
CIP-B101_Lab2A_Almustapha_Yusuf/notes/
CIP-B101_Lab2A_Almustapha_Yusuf/notes/case_note2.txt
CIP-B101_Lab2A_Almustapha_Yusuf/notes/case_note1.txt
CIP-B101_Lab2A_Almustapha_Yusuf/outputs/
CIP-B101_Lab2A_Almustapha_Yusuf/outputs/system_summary.txt
CIP-B101_Lab2A_Almustapha_Yusuf/screenshots/
CIP-B101_Lab2A_Almustapha_Yusuf/scripts/
CIP-B101_Lab2A_Almustapha_Yusuf/scripts/lab2a_sys_summary.sh

(kali@kali)-[~]
$ ls -lh CIP-B101_Lab2A_Almustapha_Yusuf.zip CIP-B101_Lab2A_Almustapha_Yusuf.tar.gz
-rw-rw-r-- 1 kali kali 1.7K Aug 15 19:59 CIP-B101_Lab2A_Almustapha_Yusuf.tar.gz
-rw-rw-r-- 1 kali kali 5.4K Aug 15 19:58 CIP-B101_Lab2A_Almustapha_Yusuf.zip

(kali@kali)-[~]
$
```

Screenshot 11.2: Archive file sizes comparison (zip vs tar.gz)

Format	Size	Entries	Compression
.zip	5.4K	18 entries	ZIP compression (Deflate)
.tar.gz	1.7K	17 entries	Gzip compression

Table 5: Archive Format Comparison

Both archives successfully captured all workspace contents including the evidence files, notes, scripts, and outputs. The tar.gz archive (1.7K) achieved significantly better compression than the zip archive (5.4K), approximately three times smaller. This difference is because gzip typically achieves better compression ratios than the ZIP Deflate algorithm for text-based content. The verbose output from both commands confirmed that all 6 directories and 7 files were included in each archive, with the tar command listing each entry as it was added.

13. Report Questions and Answers

Q1: What is a Linux inode and why is it important?

An inode (index node) is a data structure in a Unix/Linux file system that stores metadata about a file, excluding its name and actual data content. Each inode contains information such as the file size, owner (UID/GID), permissions (mode), timestamps (access, modification, change, and birth), and most critically, pointers to the data blocks on disk where the file content is stored. Every file in the filesystem is associated with exactly one inode, identified by a unique inode number. Inodes are extremely important in digital forensics for several reasons. First, when a file is deleted, only the directory entry (mapping the filename to the inode) is removed; the inode and data blocks may remain intact until overwritten, which is the basis for file recovery techniques. Second, inode timestamps can reveal when files were created, last accessed, and last modified, which is crucial for establishing timelines in investigations. Third, the number of inodes is finite, and inode exhaustion (even with available disk space) can indicate certain types of attacks or system misuse. Forensic tools like The Sleuth Kit operate directly on inodes to recover deleted files and analyze filesystem activity.

Q2: What is the difference between an absolute path and a relative path?

An absolute path specifies the complete location of a file or directory starting from the root directory (/), providing an unambiguous reference that works from any location in the filesystem. For example, /home/kali/evidence/lab2a_evidence.txt is an absolute path that always refers to the same file regardless of the user's current working directory. A relative path, on the other hand, specifies the location relative to the current working directory, using dot (.) for the current directory and double dot (..) for the parent directory. For example, evidence/lab2a_evidence.txt is a relative path that only resolves correctly if the user is in /home/kali (or another directory containing an evidence subdirectory). In forensic documentation, absolute paths are strongly preferred because they eliminate any ambiguity about file locations, ensuring that evidence references are precise and reproducible regardless of the reviewer's current directory context.

Q3: Why does Linux say 'everything is a file'? Give two examples.

In Linux and Unix systems, the 'everything is a file' philosophy means that the operating system exposes most system resources and interfaces through the file abstraction, allowing them to be accessed using standard file operations like open, read, write, and close. This design simplifies the system architecture and provides a uniform interface for interacting with diverse system resources. Two examples from the /dev and /proc directories illustrate this concept. First, in /dev, physical hardware devices are represented as files: /dev/sda represents the first SCSI/SATA disk, and

reading from or writing to this file directly accesses the raw disk data, which is exactly how forensic disk imaging tools like dd work. Second, in /proc, kernel and process information is exposed as virtual files: /proc/cpuinfo contains CPU information as readable text, and /proc//maps shows the memory map of a process. These are not real files on disk but are generated dynamically by the kernel when accessed, demonstrating how the file abstraction extends to runtime system information.

Q4: What is the purpose of the PATH variable?

The PATH variable is an environment variable containing a colon-separated list of directories that the shell searches when a command is entered without a full path. For example, when a user types 'ls', the shell checks each directory in PATH from left to right until it finds an executable named 'ls', then runs it. This mechanism is why commands like ls, grep, cat, and cd can be executed from any directory without typing their full path (e.g., /usr/bin/ls). Without PATH, users would need to remember and type the complete absolute path of every command they want to run. PATH can be modified using the export command, and adding custom directories to PATH allows user-created scripts and programs to be invoked just like system commands. In Task 10 of this lab, we demonstrated this by adding the scripts directory to PATH, which enabled lab2a_sys_summary.sh to be executed from /tmp without specifying its full path.

Q5: Which grep command identified failed login evidence?

The command `grep -n` with the pattern `failed` on `evidence/activity_log.txt` identified the failed login evidence. This command searched the activity log for the literal string 'failed' and displayed the matching line with its line number (line 4): `'2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15'`. The `-n` flag added line numbers, which is useful for documenting the exact location of evidence within a file. The output shows that user 'admin' attempted to log in at 10:05 from IP address 192.168.56.15 but the login failed, which in a real investigation could indicate a brute-force attack, credential stuffing attempt, or unauthorized access attempt by someone using the admin account.

Q6: Why should a forensic trainee avoid scanning unauthorized external systems?

Forensic trainees must never scan or probe unauthorized external systems for several important legal and ethical reasons. First, unauthorized network scanning is illegal in many jurisdictions under computer fraud and abuse laws, and can result in criminal prosecution, civil liability, and professional sanctions. Even seemingly innocuous actions like port scanning can be interpreted as reconnaissance preceding a cyberattack. Second, scanning external systems without authorization violates the principle of consent, which is a foundational ethical requirement in both cybersecurity and digital forensics. Third, unauthorized scanning can trigger security alerts at the target organization, potentially causing operational disruption and damaging the reputation of the trainee's institution. Fourth, in forensic contexts, evidence obtained through unauthorized means may be inadmissible in court and can compromise an entire investigation. Legitimate forensic work is always conducted on systems that the investigator has explicit legal authority to examine, such as owned systems, client-authorized systems, or systems covered by a court order or search warrant.

Q7: What is the difference between zip and tar.gz in this lab context?

Both zip and tar.gz create compressed archives, but they differ in several important ways relevant to forensic work. The zip format combines archiving and compression in a single tool, producing .zip files that are universally supported across Windows, macOS, and Linux without additional software. In this lab, the zip archive was 5.4K. The tar.gz format is a two-step process: tar bundles files into a single archive (preserving Unix permissions, ownership, and timestamps), and then gzip

compresses the archive. In this lab, the tar.gz archive was 1.7K, significantly smaller than the zip due to gzip's superior compression ratio on text-based content. For forensic evidence handling on Linux systems, tar.gz is generally preferred because it better preserves file metadata (permissions, ownership, symbolic links, and timestamps), which can be important evidence in itself. However, zip is more convenient when evidence needs to be shared with Windows-based reviewers who may not have tar tools readily available.

14. Conclusion

This laboratory exercise successfully demonstrated all fundamental Linux command-line skills required for digital forensic work. Through eleven structured tasks, the lab covered the complete workflow from workspace preparation through evidence archiving. Key competencies developed include navigating the filesystem hierarchy with absolute and relative paths, understanding inode metadata and its forensic significance, managing files with copy/rename/delete operations, searching evidence content with grep patterns including regex and case-insensitive matching, discovering files with tree and find, checking network configuration safely using localhost, writing executable shell scripts that automate system information collection, and packaging evidence into compressed archives using both zip and tar.gz formats.

The skills practiced in this lab directly translate to real-world forensic operations. The grep searches on the activity log mirror the process of analyzing authentication logs during incident response. The shell script creation demonstrates how forensic tools automate repetitive collection tasks. The inode examination provides the foundation for understanding file recovery techniques. The network commands establish the practice of documenting system network context while maintaining ethical boundaries. Together, these skills form the operational foundation upon which more advanced forensic techniques such as disk imaging, memory analysis, and network forensics are built in subsequent coursework.